Section 12: Recursion and Backtracking

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

C11: DSA Foundations for Interviews

Section 12: Recursion and Backtracking

- SLIDE 110: Section Intro, Recursion and Backtracking
- SLIDE 111: The Recursive Mental Model
- SLIDE 112: Anatomy of Recursion and the Call Stack
- SLIDE 113: Recursion Complexity and Common Patterns
- SLIDE 114: Backtracking: Choose, Explore, Unchoose
- SLIDE 115: Permutations
- SLIDE 115B: Subsets and Combinations
- SLIDE 116: Pruning: Worked Example with Combination Sum
- SLIDE 117: Wrong vs Right: Recursion and Backtracking Mistakes
- SLIDE 118: How to Present Backtracking in an Interview
- SLIDE 119: Interview Practice: Recursion and Backtracking
- SLIDE 120: Section Cheat Sheet

C11: DSA Foundations for Interviews

Section 12: Recursion and Backtracking

SLIDE 110: Section Intro, Recursion and Backtracking

Section 12: Recursion and Backtracking

Recursion is a function calling itself on a smaller version of the same problem. Backtracking is recursion with an undo step: build a solution piece by piece, and when a choice leads to a dead end, undo it and try the next option. Together, these two techniques are the foundation for tree traversal, graph search, and dynamic programming, all of which are covered in C6.

This section receives the largest time allocation in the course for a deliberate reason: teaching recursion too quickly produces students who can recite the definition but cannot apply it to a new problem. The gap between “I understand recursion conceptually” and “I can write a recursive solution to a problem I have never seen” is where most freshers fail in interviews. This section closes that gap.

What We’ll Cover:

1. The recursive mental model: how to think about recursion without tracing every call
2. Anatomy of a recursive function: base case, recursive case, combining results
3. Recursion complexity: why some recursive solutions are O(n) and others are O(2^n)
4. The backtracking template: choose, explore, unchoose
5. Generation problems: permutations, subsets, and combinations
6. Pruning: cutting branches from the search tree to avoid wasted work

Where Recursion and Backtracking Connect to Other Sections:

Connection	Section	How It Applies
Merge Sort is recursive divide-and-conquer	Section 9: Sorting	Split, sort halves recursively, merge
Linked list operations can be recursive	Section 11: Linked Lists	Reverse, merge, and traverse recursively
Tree traversals are recursive by nature	C6: Trees	Inorder, preorder, postorder are all recursion
Graph DFS is recursion on adjacency lists	C6: Graphs	Explore neighbors recursively, mark visited
Dynamic programming optimizes overlapping recursion	C6: DP	Memoize recursive subproblems, or convert to iterative DP

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 111: The Recursive Mental Model

The Biggest Obstacle to Learning Recursion

Most freshers try to learn recursion by mentally tracing every recursive call. They simulate the call stack in their head, trying to follow exactly which call returns what and when. This approach works for factorial(3) but collapses for anything with more than four or five levels. It is the wrong approach.

The "Trust the Recursion" Principle

Instead of tracing every call, assume the recursive call returns the correct answer for the smaller problem. Your only job is:

1. What is the base case? (When do I stop recursing?)
2. How do I combine the recursive result with the current step?

If those two pieces are correct, the recursion works. You do not need to verify every intermediate call manually.

Example: Sum of an Array

"Find the sum of all elements in an array."

Thinking process:

- Base case: if the array is empty, the sum is 0.
- Recursive case: the sum is the first element plus the sum of the remaining elements.
- Trust that summing the remaining elements returns the correct answer.

```java
public int arraySum(int[] nums, int index) {
    if (index == nums.length) {
        return 0;
    }
    return nums[index] + arraySum(nums, index + 1);
}
```

You do not need to trace arraySum(nums, 1) returning arraySum(nums, 2) returning arraySum(nums, 3)... You need to verify: does the base case handle the end? (Yes, returns 0.) Does the recursive case correctly combine one element with the result for the rest? (Yes, adds nums[index] to the sum of the remainder.) Both correct, so the function is correct.

Example: Reverse a String

- Base case: empty string or single character returns itself.
- Recursive case: last character plus the reverse of everything before it.

```java
public String reverse(String s) {
    if (s.length() <= 1) {
        return s;
    }
    return s.charAt(s.length() - 1) + reverse(s.substring(0, s.length() - 1));
}
```

Trust that reverse(s.substring(0, s.length() - 1)) correctly reverses the shorter string. Your job: put the last character at the front and attach the reversed remainder. Correct.

Why Freshers Struggle

The instinct to trace everything comes from sequential thinking: “execute line 1, then line 2, then line 3.” Recursion requires declarative thinking: “assume the subproblem is solved, and describe how to use its answer.” This is a different mode of reasoning, and it takes practice. The problems in this section build that practice.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 112: Anatomy of Recursion and the Call Stack

Three Components Every Recursive Function Needs

Every recursive function must have exactly these three parts:

Component	Purpose	What Goes Wrong Without It
Base case	When to stop recursing	Infinite recursion, StackOverflowError
Recursive case	How to reduce the problem	No progress toward the base case
Combining step	How to use the recursive result	Correct recursion but wrong final answer

Worked Example: Factorial

n! = n * (n-1) * (n-2) * ... * 1. Base case: 0! = 1 and 1! = 1.

```java
public int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

Base case: n <= 1 returns 1. Recursive case: multiply n by factorial of (n-1). Combining step: the multiplication itself.

The Call Stack: What Happens in Memory

When factorial(4) executes, each recursive call pushes a new frame onto the call stack. The frames unwind in reverse order once the base case is reached.

```
Call:    factorial(4) → needs factorial(3)
Call:    factorial(3) → needs factorial(2)
Call:    factorial(2) → needs factorial(1)
Call:    factorial(1) → base case, returns 1

Return:  factorial(2) → 2 * 1 = 2
Return:  factorial(3) → 3 * 2 = 6
Return:  factorial(4) → 4 * 6 = 24
```

Stack depth = n. Each frame uses O(1) space, so total space is O(n). This is the space cost of recursion that interviewers expect you to mention. Even if the function uses no auxiliary data structures, the call stack itself consumes O(n) memory.

Recursion vs Iteration: When to Prefer Each

Consideration	Recursion	Iteration
Space	O(depth) call stack	O(1) if no stack needed
Clarity	Cleaner for trees, graphs, subsets, divide-and-conquer	Cleaner for simple loops, accumulation
Stack overflow risk	Yes, for depth > ~10,000 in Java	No
Interview preference	Required for backtracking, trees, graphs	Preferred for linked list reversal, simple traversals

The practical rule: prefer iteration when the iterative version is clean and space matters (linked list reversal: iterative O(1) space vs recursive O(n) stack). Prefer recursion when the problem has natural recursive structure (trees, subsets, permutations, divide-and-conquer) and the iterative version would require manually managing a stack.

Interview Tip: When you write a recursive solution, always state the space complexity of the call stack. “Time is O(n), and space is O(n) due to the recursion stack depth.” Interviewers mark candidates down for forgetting the stack space. If the interviewer asks about stack overflow on large inputs, mention that the iterative version avoids this.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 113: Recursion Complexity and Common Patterns

Recursive complexity depends on two factors: how many recursive calls each function makes (branches) and how the input size shrinks at each level (reduction). These two factors determine the shape of the recursion tree.

Quick Reference Table:

Pattern	Branches	Input Reduction	Total Calls	Complexity	Examples
Linear	1	Reduce by 1	n	O(n)	Factorial, linked list traversal
Halving	1	Divide by 2	log n	O(log n)	Binary search (recursive)
Divide-and-conquer	2	Divide by 2	n (total work across levels)	O(n log n)	Merge sort
Exponential (2-branch)	2	Reduce by 1	2^n	O(2^n)	Subsets, naive Fibonacci
Factorial	n, n-1, n-2...	Reduce by 1	n!	O(n!)	Permutations

How to Read the Table

Linear recursion: one call per level, n levels deep. Total calls = n. Think of it as a chain.

Divide-and-conquer: two calls per level, each on half the input. The tree has log n levels, and each level does O(n) total work across all calls. Total: O(n log n). Think of it as a balanced binary tree.

Exponential: two calls per level, each reducing input by 1 (not half). The tree has n levels with 2^n leaves. Every possible binary choice at each position is explored. Think of it as a full binary tree of depth n.

Worked Example: Naive Fibonacci

```
public int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}
```

Two branches at every call, each reducing n by 1 or 2. The recursion tree has roughly 2^n nodes. fib(5) makes 15 calls. fib(40) makes over 300 million calls.

The problem: massive duplication. fib(3) is computed from fib(5), from fib(4), and from several other branches. The same subproblem is solved over and over.

The Fix: Memoization

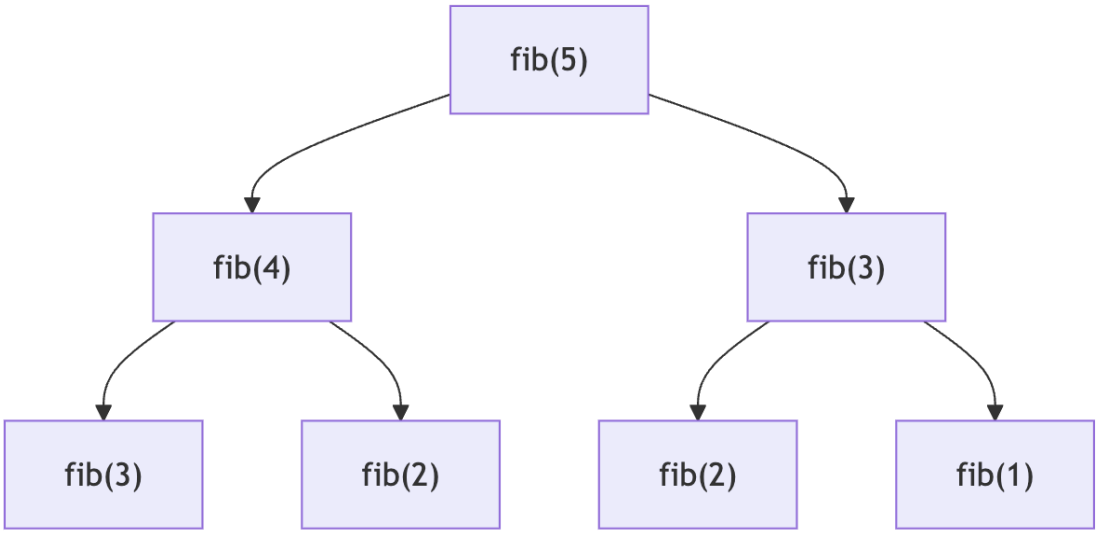
Store computed results and reuse them. Each fib(k) is computed once and looked up afterward.

```
public int fib(int n) {
    int[] memo = new int[n + 1];
    Arrays.fill(memo, -1);
```

```
    return fibHelper(n, memo);
}

private int fibHelper(int n, int[] memo) {
    if (n <= 1) {
        return n;
    }
    if (memo[n] != -1) {
        return memo[n];
    }
    memo[n] = fibHelper(n - 1, memo) + fibHelper(n - 2, memo);
    return memo[n];
}
```

Time drops from O(2^n) to O(n). Space is O(n) for the memo array plus O(n) for the call stack. This technique, memoization, is the bridge from recursion to dynamic programming (covered in C6).



Notice fib(3) and fib(2) are each computed multiple times in the naive version. Memoization computes each once.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 114: Backtracking: Choose, Explore, Unchoose

What Backtracking Is

Backtracking is recursion with an undo step. You build a solution one choice at a time. After exploring what happens with a particular choice, you undo it and try the next option. This explores all possible solutions

systematically.

The template has three steps at each decision point:

1. **Choose:** Add an element to the current partial solution.
2. **Explore:** Recurse with the remaining options.
3. **Unchoose:** Remove the element, restoring state for the next option.

The Java Template:

```
public void backtrack(List<List<Integer>> result, List<Integer> current, int[] optio
ns, ...) {
    if (reachedGoal(current)) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int i = startOrCondition; i < options.length; i++) {
        current.add(options[i]);                    // choose
        backtrack(result, current, options, ...);   // explore
        current.remove(current.size() - 1);         // unchoose
    }
}
```

Why the Unchoose Step Matters

Without the unchoose step, the current list carries state from one branch into the next. Each recursive branch sees the leftovers of every previous branch, producing wrong results.

Consider generating subsets of [1, 2, 3]. Without unchoose:

```
Branch 1 adds 1, then 2, then 3. current = [1, 2, 3].
Branch 2 starts, but current is still [1, 2, 3] instead of empty.
Every subsequent subset is wrong.
```

With unchoose: after exploring [1, 2, 3], the function removes 3, then removes 2, then removes 1, restoring current to [] before starting the next top-level branch.

Why result.add(new ArrayList<>(current)) Uses a Copy

The current list is mutated throughout the recursion. If you add current directly to result, every entry in result points to the same list object. When current changes, all previously added entries change with it. At the end, result contains n copies of the same (final) state.

new ArrayList<>(current) creates a snapshot. That snapshot is independent of future mutations to current.

When to Recognize Backtracking:

Problem Signal	What It Means
“Generate all permutations”	Every ordering of all elements
“Generate all subsets”	Every possible subset including empty
“Find all combinations that sum to K”	All groups meeting a constraint
“All valid configurations”	Placement problems (N-Queens)

Problem Signal	What It Means
“All paths from source to target”	Graph/tree path enumeration

If the problem asks you to enumerate all possibilities rather than find one optimal solution, backtracking is almost always the technique.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 115: Permutations

Problem: Generate All Permutations of Distinct Integers (LeetCode 46)

Given [1, 2, 3], produce all 6 orderings: [1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1].

Why Permutations Use a “Used” Array, Not a Start Index

In permutations, order matters and every element must appear exactly once. At each position, you can choose any element that has not already been used. A boolean[] used array tracks which elements are currently in the partial solution.

This is different from subsets and combinations, where a start index prevents revisiting earlier elements because order does not matter in those problems.

Full Java Solution:

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    boolean[] used = new boolean[nums.length];
    backtrack(result, new ArrayList<>(), nums, used);
    return result;
}

private void backtrack(List<List<Integer>> result, List<Integer> current,
                       int[] nums, boolean[] used) {
    if (current.size() == nums.length) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;

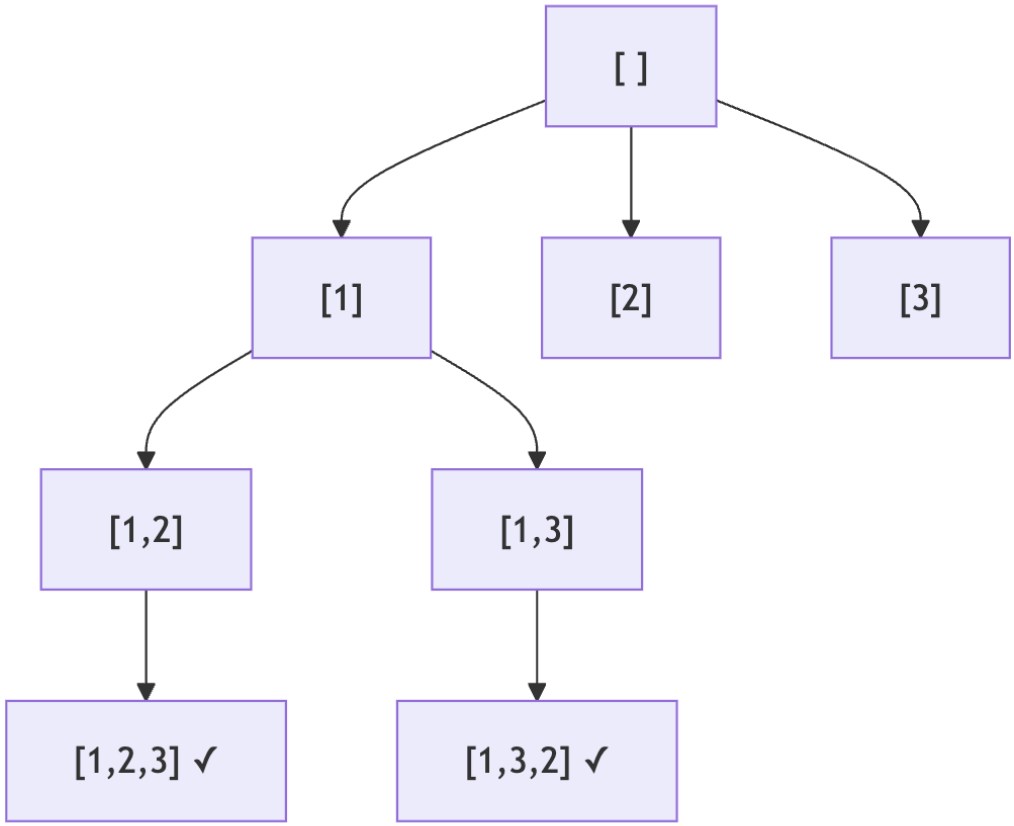
        current.add(nums[i]);          // choose
        used[i] = true;

        backtrack(result, current, nums, used);  // explore

        current.remove(current.size() - 1);      // unchoose
        used[i] = false;
```

```
        }
    }
```

Decision Tree Trace for [1, 2, 3]:



The full tree has 3 branches at level 1, 2 branches at level 2, and 1 branch at level 3. Total leaves: 3! = 6 permutations.

Complexity:

Metric	Value	Why
Time	O(n * n!)	n! permutations, each takes O(n) to copy into result
Space	O(n)	Recursion depth n, plus the used array of size n
Number of results	n!	3! = 6, 4! = 24, 5! = 120, 10! = 3,628,800

Step-by-Step Trace of First Two Permutations:

Call Depth	current	used[]	Action
0	[]	[F, F, F]	Try i=0: choose 1
1	[1]	[T, F, F]	i=0 used, skip. Try i=1: choose 2
2	[1, 2]	[T, T, F]	i=0,1 used. Try i=2: choose 3

Call Depth	current	used[]	Action
3	[1, 2, 3]	[T, T, T]	Size == n. Add copy to result. Return.
2	[1, 2]	[T, T, F]	Unchoose 3. No more options at this level. Return.
1	[1]	[T, F, F]	Unchoose 2. Try i=2: choose 3
2	[1, 3]	[T, F, T]	Try i=1: choose 2
3	[1, 3, 2]	[T, T, T]	Size == n. Add copy to result. Return.

The pattern continues, producing all 6 permutations.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 115B: Subsets and Combinations

Subsets: Generate All Subsets of Distinct Integers (LeetCode 78)

Given [1, 2, 3], produce all 8 subsets: [], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3].

Why Subsets Use a Start Index, Not a Used Array

In subsets, order does not matter. [1, 2] and [2, 1] are the same subset. A start index ensures you only look forward, never backward, preventing duplicate subsets.

Full Java Solution:

```java
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(result, new ArrayList<>(), nums, 0);
    return result;
}

private void backtrack(List<List<Integer>> result, List<Integer> current,
                       int[] nums, int start) {
    result.add(new ArrayList<>(current));

    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);                          // choose
        backtrack(result, current, nums, i + 1);       // explore (start = i + 1)
        current.remove(current.size() - 1);            // unchoose
    }
}
```

Two differences from permutations. First, every node in the tree is a valid subset (not just the leaves), so result.add happens before the loop, not inside a size check. Second, the loop starts at start instead of 0, and the recursive call passes i + 1, ensuring each element is considered at most once.

Trace for [1, 2, 3]:

Call	start	current	Action
1	0	[]	Add [] to result. Loop i=0,1,2
2	1	[1]	Add [1]. Loop i=1,2
3	2	[1,2]	Add [1,2]. Loop i=2
4	3	[1,2,3]	Add [1,2,3]. Loop empty. Return.
3	2	[1,2]	Unchoose 3. Return.
2	1	[1]	Unchoose 2. Try i=2
3	3	[1,3]	Add [1,3]. Loop empty. Return.
...	...	...	Continue for [2], [2,3], [3]

Result: [[], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]]. Total: 2^3 = 8 subsets.

Combinations: Choose K Elements from N (LeetCode 77)

Combinations are subsets with a size constraint. Choose(4, 2) means all subsets of size 2 from [1, 2, 3, 4].

```
public List<List<Integer>> combine(int n, int k) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(result, new ArrayList<>(), n, k, 1);
    return result;
}

private void backtrack(List<List<Integer>> result, List<Integer> current,
                       int n, int k, int start) {
    if (current.size() == k) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int i = start; i <= n; i++) {
        current.add(i);                                  // choose
        backtrack(result, current, n, k, i + 1);         // explore
        current.remove(current.size() - 1);              // unchoose
    }
}
```

Same as subsets, except: add to result only when current.size() == k (not at every node), and return early when the size constraint is met.

Key Differences Table:

Property	Permutations	Subsets	Combinations
Order matters?	Yes	No	No
Element reuse?	No (each element once)	No (each element once)	No (each element once)
Duplicate prevention	boolean[] used array	Start index	Start index
When to add to result	current.size() == n	Every recursive call	current.size() == k
Result count	n!	2^n	C(n, k)
Loop range	0 to n-1, skip used	start to n-1	start to n

Property	Permutations	Subsets	Combinations
Time complexity	O(n * n!)	O(n * 2^n)	O(k * C(n,k))

Interview Tip: If you get a generation problem and you are unsure whether it is permutations, subsets, or combinations, ask two questions. “Does order matter?” If yes, permutations. If no, “Is there a size constraint?” If yes, combinations. If no, subsets. These two questions determine the template.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 116: Pruning: Worked Example with Combination Sum

What Pruning Is

Pruning means cutting branches from the search tree before exploring them. If you can determine that a branch cannot lead to a valid solution, skip it entirely. This does not change the results (you never miss a valid answer), but it reduces the number of recursive calls, sometimes dramatically.

Problem: Combination Sum (LeetCode 39)

Given an array of distinct positive integers and a target, find all unique combinations where the chosen numbers sum to the target. Each number can be used unlimited times.

candidates = [2, 3, 6, 7], target = 7.
Output: [[2, 2, 3], [7]].

Step 1: Understand

Unlimited reuse of each candidate. Combinations, not permutations (order does not matter, so [2, 3, 2] and [2, 2, 3] are the same). No duplicate combinations in the result.

Step 2: Identify Pattern

“Find all combinations” with a running sum constraint. Backtracking with start index (order does not matter) and the ability to reuse the current element (start at i, not i + 1).

Step 3: Plan and Implement

Sort the candidates first. When the running sum exceeds the target, stop exploring that branch and all subsequent branches (since candidates are sorted, every future candidate is at least as large).

```
public List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(candidates);
    backtrack(result, new ArrayList<>(), candidates, target, 0, 0);
    return result;
}

private void backtrack(List<List<Integer>> result, List<Integer> current,
                       int[] candidates, int target, int start, int currentSum) {
    if (currentSum == target) {
        result.add(new ArrayList<>(current));
        return;
```

```
    }

    for (int i = start; i < candidates.length; i++) {
        if (currentSum + candidates[i] > target) {
            break;
        }

        current.add(candidates[i]);                          // choose
        backtrack(result, current, candidates, target,
                  i, currentSum + candidates[i]);            // explore (i, not i+
1: reuse allowed)
        current.remove(current.size() - 1);                  // unchoose
    }
}
```

Note: the recursive call passes i (not i + 1) because each candidate can be reused. The start index still prevents [3, 2, 2] from appearing alongside [2, 2, 3], since we never look backward.

Step 4: Trace through candidates = [2, 3, 6, 7], target = 7:

Depth	current	sum	Next candidate	Action
0	[]	0	2	Choose 2
1	[2]	2	2	Choose 2
2	[2, 2]	4	2	Choose 2
3	[2, 2, 2]	6	2	6+2=8 > 7, **prune**. Try 3: 6+3=9 > 7, **prune**. Return.
2	[2, 2]	4	3	Choose 3
3	[2, 2, 3]	7		Sum == target. Add to result. Return.
2	[2, 2]	4	6	4+6=10 > 7, **prune**. Return.
1	[2]	2	3	Choose 3
2	[2, 3]	5	3	5+3=8 > 7, **prune**. Try 6: 5+6=11 > 7, **prune**. Return.
1	[2]	2	6	2+6=8 > 7, **prune**. Return.
0	[]	0	3	Choose 3
1	[3]	3	3	Choose 3
2	[3, 3]	6	3	6+3=9 > 7, **prune**. Return.
1	[3]	3	6	3+6=9 > 7, **prune**. Return.
0	[]	0	6	Choose 6
1	[6]	6	6	6+6=12 > 7, **prune**. Try 7: 6+7=13 > 7, **prune**. Return.
0	[]	0	7	Choose 7
1	[7]	7		Sum == target. Add to result. Return.

Result: [[2, 2, 3], [7]]. The prune step (break when sum + candidate > target) eliminated entire subtrees. Without pruning, the recursion would explore many more branches that could never reach the target.

Why Sorting Enables Pruning

The break statement works because the candidates are sorted. Once candidates[i] is too large, every candidate after it is also too large. Without sorting, you would need to use continue (skip this one, try the next) instead of break (stop all remaining), which prunes fewer branches.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 117: Wrong vs Right: Recursion and Backtracking Mistakes

Mistake 1: Forgetting the Base Case

Wrong:

```
public int factorial(int n) {
    return n * factorial(n - 1);
}
```

No base case. For any positive n, this recurses until the stack overflows. For n = 0, it recurses into negative numbers forever.

Right:

```
public int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

Mistake 2: Not Undoing the Choice

Wrong:

```
current.add(nums[i]);
backtrack(result, current, nums, i + 1);
// forgot current.remove(current.size() - 1)
```

The next iteration of the for loop sees current still containing nums[i]. State leaks between branches, and every subsequent path is polluted.

Right:

```
current.add(nums[i]);
backtrack(result, current, nums, i + 1);
current.remove(current.size() - 1);
```

Mistake 3: Adding the Mutable List Directly to Result

Wrong:

```
if (current.size() == nums.length) {
    result.add(current);    // adds reference, not snapshot
```

```
}
```

Every entry in result points to the same list object. By the time recursion finishes, current is empty, and result contains n! copies of [].

Right:

```
result.add(new ArrayList<>(current));    // snapshot copy
```

Mistake 4: Wrong Deduplication Mechanism

Wrong: using a boolean[] used array for subsets. This allows [2, 1] and [1, 2] as separate subsets because each uses different elements in a different order.

Right: use a start index for subsets and combinations (order does not matter). Use boolean[] used only for permutations (order matters).

Problem Type	Deduplication	Why
Permutations	boolean[] used	Order matters, need to try every position
Subsets	Start index (i + 1)	Order irrelevant, only look forward
Combinations	Start index (i + 1)	Order irrelevant, only look forward
Combination Sum (reuse)	Start index (i, not i + 1)	Reuse allowed, but no backward look

Mistake 5: Not Sorting Before Pruning

Wrong:

```
if (currentSum + candidates[i] > target) {
    continue;    // skips this candidate but tries the next
}
```

Without sorting, a large candidate might appear before a small one. Skipping the large one but trying the next (smaller) one is correct but misses the optimization: with sorted input, you can break instead of continue, because all remaining candidates are at least as large.

Right: sort the input. Use break instead of continue when pruning on a sorted sequence.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 118: How to Present Backtracking in an Interview

How to Introduce Backtracking

“This problem asks me to generate all valid combinations/permutations/subsets. This is a backtracking problem. I’ll build solutions incrementally, choosing one element at a time and undoing the choice after exploring that branch to try alternatives.”

How to Explain the Template

“My backtrack function has three phases. Choose: add an element to the current partial solution. Explore: recurse with the remaining options. Unchoose: remove the element to restore state for the next iteration. The

base case adds the current solution to the result when it meets the goal."

How to Explain Pruning

"I sort the input first so I can terminate early. When the running sum plus the next candidate exceeds the target, I break out of the loop because every remaining candidate is at least as large and would also exceed the target. This prunes entire subtrees without affecting correctness."

Drawing the Decision Tree

One of the most effective things you can do in a backtracking interview is draw a small decision tree. Even a 2-3 level sketch with 4-6 nodes shows the interviewer you understand the recursive structure.

For subsets of [1, 2]:

```
Level 0:  []
Level 1:  [1]        [2]
Level 2:  [1,2]
All nodes: [], [1], [1,2], [2] → 4 subsets = 2^2
```

For the interviewer, this sketch proves three things: you understand the branching structure, you know where results are collected (every node for subsets, leaves only for permutations), and you see how the start index prevents duplicates.

When Backtracking is Not the Answer

Backtracking explores all possibilities. Its complexity is exponential (2^n for subsets, n! for permutations). If the problem asks for a count rather than enumeration, or asks for the optimal solution rather than all solutions, consider:

Problem Asks For	Technique	Why Not Backtracking
Count of valid arrangements	DP or math formula	Backtracking generates all, then counts. DP counts directly.
One optimal solution	DP or greedy	Backtracking explores all, returns best. DP finds optimal directly.
Whether a solution exists	DP or backtracking with early return	Backtracking can work but may be slower
All valid solutions	Backtracking	This is where backtracking is the right tool

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 119: Interview Practice: Recursion and Backtracking

Practice Prompt: "Generate all subsets of a given set of distinct integers."

How to Walk Through This in an Interview:

Step 1: Understand (30 seconds)

"Given an array of distinct integers, I need to produce every possible subset, including the empty set and the full set. For [1, 2, 3], that's 2^3 = 8 subsets. No duplicates in the input, so I don't need to worry about duplicate subsets."

Step 2: Identify Pattern (15 seconds)

"This is a generation problem where order doesn't matter. [1, 2] and [2, 1] are the same subset. I'll use backtracking with a start index to avoid revisiting earlier elements."

Step 3: Plan (30 seconds)

"At each recursive call, I add the current partial solution to the result because every partial solution is a valid subset. Then I iterate from the start index forward, choosing each element, recursing with start = i + 1, and unchoosing. The base case is implicit: when start exceeds the array length, the loop doesn't execute and the function returns."

Step 4: Implement

```java
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(result, new ArrayList<>(), nums, 0);
    return result;
}

private void backtrack(List<List<Integer>> result, List<Integer> current,
                       int[] nums, int start) {
    result.add(new ArrayList<>(current));

    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);
        backtrack(result, current, nums, i + 1);
        current.remove(current.size() - 1);
    }
}
```

Step 5: Verify

"For [1, 2, 3]: start at [], add it. Choose 1, recurse. Add [1]. Choose 2, recurse. Add [1,2]. Choose 3, recurse. Add [1,2,3]. Unwind, add [1,3], [2], [2,3], [3]. Total 8 subsets. Time is O(n * 2^n) because there are 2^n subsets and each takes O(n) to copy. Space is O(n) for the recursion stack."

Common Interviewer Follow-ups:

Follow-up	How to Handle
"What if the input has duplicates?"	Sort the input, skip duplicates in the for loop: if (i > start && nums[i] == nums[i-1]) continue
"Can you do this iteratively?"	Yes, start with [[]], for each num, add num to copies of all existing subsets. O(n * 2^n).
"What's the space complexity?"	O(n) for recursion depth. O(n * 2^n) if counting the output. Clarify which the interviewer means.

Common Fresher Mistakes on Subsets:

Mistake	Consequence	Fix
Using boolean[] used instead of start index	Generates [1,2] and [2,1] as separate subsets	Start index prevents backward revisiting
Adding current without copying	All entries in result point to same empty list	new ArrayList<>(current)

Mistake	Consequence	Fix
Only adding at leaves (current.size() == n)	Misses all subsets shorter than n	Add at every recursive call
Not handling empty input	Crash or missing empty set	Empty loop produces just [[]], which is correct

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 120: Section Cheat Sheet

Recursion Anatomy:

Every recursive function: base case (when to stop) + recursive case (how to reduce) + combining step (how to use the result). If any of the three is missing, the function is broken.

Recursion Complexity Quick Reference:

Pattern	Branches	Depth	Total Calls	Examples
Linear (1 branch, reduce by 1)	1	n	O(n)	Factorial, array sum, linked list traversal
Halving (1 branch, divide by 2)	1	log n	O(log n)	Recursive binary search
Divide-and-conquer (2 branches, halve)	2	log n	O(n log n) total work	Merge sort, quick sort (avg)
Exponential (2 branches, reduce by 1)	2	n	O(2^n)	Subsets, naive Fibonacci
Factorial (n branches, reduce by 1)	n..1	n	O(n!)	Permutations

Backtracking Template:

```
void backtrack(result, current, options, ...) {
    if (goalReached) {
        result.add(new ArrayList<>(current));   // always copy
        return;
    }
    for (int i = start; i < options.length; i++) {
        current.add(options[i]);                // choose
        backtrack(result, current, options, ...); // explore
        current.remove(current.size() - 1);     // unchoose
    }
}
```

Generation Problems Decision Table:

Problem	Order Matters?	Deduplication	Add to Result When	Loop Starts At	Recurse With	Count
Permutations	Yes	boolean[] used	current.size() == n	0 (skip used)	same start, toggle used	n!
Subsets	No	Start index	Every call	start	i + 1	2^n
Combinations	No	Start index	current.size() == k	start	i + 1	C(n,k)
Combination Sum	No (reuse ok)	Start index	sum == target	start	i (reuse)	varies

Pruning Checklist:

1. Sort the input before recursion
2. Check constraint before recursing (if sum + candidate > target, break)
3. Use break (not continue) when input is sorted, since remaining candidates are all larger
4. Skip duplicates: if (i > start && nums[i] == nums[i-1]) continue

Critical Mistakes to Avoid:

Mistake	Symptom	Fix
No base case	StackOverflowError	Define termination condition first
No unchoose step	Wrong results, state leaks	Remove after recursive call returns
result.add(current) without copy	All entries identical at end	new ArrayList<>(current)
used[] for subsets	Duplicate subsets ([1,2] and [2,1])	Start index instead
Start index for permutations	Missing permutations	used[] array instead

How This Connects to Other Sections:

This Section's Concept	Connects To	How
Recursion complexity (2^n, n!)	Section 2: Complexity	Branching factor and depth determine complexity class
Memoization of overlapping subproblems	C6: Dynamic Programming	DP = recursion + memoization, or bottom-up table filling
Tree traversals are recursion	C6: Trees	Preorder/inorder/postorder are recursive by nature
Graph DFS is recursion with visited tracking	C6: Graphs	DFS is backtracking on a graph adjacency structure
Merge Sort is recursive divide-and-conquer	Section 9: Sorting	Split, recurse on halves, merge results
Recursive linked list operations	Section 11: Linked Lists	Reverse and merge can be recursive (but iterative preferred)
Backtracking decision tree structure	Section 2: Complexity	Counting nodes in the decision tree gives time complexity

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina